

**LAPORAN KECERDASAN BUATAN (AI)  
SHORTEST PATH WILAYAH UNIB**



Disusun Oleh:

Nama : M Iqbal Zafarullah

NPM : G1A024007

Kelas : A1

Dosen Pengampu :

Ir. Arie Vatesia, S.T., M.T.I., Ph.D., IPP

**PROGRAM STUDI INFORMATIKA  
FAKULTAS TEKNIK  
UNIVERSITAS BENGKULU**

**2026**

## **DAFTAR ISI**

### **BAB I PENDAHULUAN**

- 1.1 Latar Belakang Masalah
- 1.2 Rumusan Masalah
- 1.3 Tujuan Pengembangan KibokGO

### **BAB II TINJAUAN PUSTAKA DAN ARSITEKTUR SISTEM**

- 2.1 Arsitektur Website KibokGO
- 2.2 Cara Kerja Sistem Routing
- 2.3 Penjelasan Algoritma Dijkstra
- 2.4 Teknologi yang Digunakan

### **BAB III IMPLEMENTASI DAN PEMBAHASAN**

- 3.1 Implementasi Graf Kampus UNIB
- 3.2 Implementasi Algoritma Dijkstra
- 3.3 Penjelasan File-File Utama Proyek
- 3.4 Pengujian Pencarian Rute
- 3.5 Analisis Hasil Pengujian

### **BAB IV PANDUAN PENGGUNAAN**

- 4.1 Gambaran Umum Antarmuka
- 4.2 Panduan Penggunaan Fitur Pencarian Rute
- 4.3 Fitur Tampilan Jaringan Graf
- 4.4 Fitur Rute Cadangan (Detour)
- 4.5 Fitur Mode Peta dan Tema
- 4.6 Dokumentasi Antarmuka dan Fitur Sistem

### **BAB V KESIMPULAN DAN SARAN**

- 5.1 Kesimpulan
- 5.2 Saran Pengembangan

# **BAB I**

## **PENDAHULUAN**

### **1.1 Latar Belakang Masalah**

Universitas Bengkulu (UNIB) merupakan salah satu perguruan tinggi negeri terkemuka di Provinsi Bengkulu yang memiliki area kampus yang luas dengan puluhan gedung perkuliahan, fasilitas umum, dan area parkir yang tersebar di berbagai lokasi. Kondisi ini seringkali menimbulkan kesulitan bagi mahasiswa baru, tamu, maupun civitas akademika dalam menemukan lokasi tujuan dengan cepat dan efisien di lingkungan kampus.

Selama ini, navigasi di lingkungan Kampus UNIB masih mengandalkan cara konvensional seperti bertanya kepada orang sekitar atau mengandalkan peta statis yang tidak interaktif. Metode ini memiliki banyak kelemahan, antara lain tidak dapat memberikan informasi jalur terpendek secara real-time, tidak mampu beradaptasi ketika terdapat hambatan jalan, dan tidak tersedia dalam genggam tangan pengguna kapan saja dan di mana saja.

Perkembangan teknologi informasi khususnya di bidang Artificial Intelligence (AI) dan sistem informasi geografis membuka peluang untuk membangun solusi navigasi kampus yang cerdas dan adaptif. Algoritma pencarian jalur terpendek seperti Algoritma Dijkstra telah terbukti mampu menyelesaikan permasalahan pencarian rute optimal pada jaringan graf berbobot dengan efisiensi tinggi. Dengan memanfaatkan teknologi web modern, pustaka peta interaktif Leaflet.js, dan data spasial OpenStreetMap, sebuah sistem navigasi kampus berbasis web dapat dibangun dan diakses langsung dari browser tanpa memerlukan instalasi aplikasi tambahan.

Berdasarkan latar belakang tersebut, dikembangkanlah KibokGO UNIB, sebuah sistem navigasi kampus berbasis web yang mengimplementasikan Algoritma Dijkstra secara client-side untuk menentukan jalur terpendek antar titik-titik lokasi penting di Kampus UNIB, serta dilengkapi dengan fitur penghindaran rute terblokir (detour routing) yang memungkinkan sistem beradaptasi secara dinamis terhadap kondisi jalan yang berubah.

### **1.2 Rumusan Masalah**

Berdasarkan latar belakang yang telah diuraikan di atas, rumusan masalah pada proyek ini adalah sebagai berikut:

- 1. Bagaimana memodelkan jaringan jalan Kampus UNIB sebagai graf berbobot yang akurat berdasarkan koordinat GPS aktual setiap gedung dan fasilitas kampus?
- 2. Bagaimana merancang dan mengimplementasikan Algoritma Dijkstra secara client-side pada browser untuk menentukan jalur terpendek antar dua titik lokasi di Kampus UNIB?
- 3. Bagaimana membangun fitur detour routing yang mampu mencari dan menampilkan rute cadangan secara otomatis ketika pengguna menandai suatu jalur sebagai terblokir?
- 4. Bagaimana merancang antarmuka web yang modern, responsif, dan intuitif untuk mempermudah pengguna dalam mengoperasikan sistem navigasi kampus?
- 5. Bagaimana memvisualisasikan struktur graf jaringan jalan kampus dan proses komputasi Algoritma Dijkstra secara transparan kepada pengguna melalui antarmuka web?

### **1.3 Tujuan Pengembangan KibokGO**

Tujuan yang ingin dicapai dari pengembangan sistem KibokGO UNIB ini adalah:

- 1. Mengimplementasikan Algoritma Dijkstra sebagai mesin inti pencarian jalur terpendek yang berjalan secara mandiri di sisi browser klien (client-side JavaScript).
- 2. Membangun model graf berbobot Kampus UNIB berdasarkan koordinat GPS aktual seluruh gedung dan fasilitas, direpresentasikan dalam bentuk adjacency list yang efisien.
- 3. Mengintegrasikan fitur rute cadangan (detour routing) yang mampu menemukan jalur alternatif secara dinamis ketika suatu rute utama diblokir.
- 4. Menyajikan antarmuka web interaktif berbasis HTML, CSS, JavaScript, dan Leaflet.js yang memungkinkan visualisasi peta, pencarian rute, dan pengelolaan kondisi jalan secara intuitif.
- 5. Menyediakan fitur transparansi akademis berupa tampilan log perhitungan langkah-demi-langkah Algoritma Dijkstra dan overlay visual jaringan graf kampus pada peta interaktif.

## BAB II

### TINJAUAN PUSTAKA DAN ARSITEKTUR SISTEM

#### 2.1 Arsitektur Website KibokGO

KibokGO UNIB adalah sistem navigasi berbasis web bertipe Single Page Application (SPA) yang dirancang untuk membantu mahasiswa, staf, dan pengunjung Universitas Bengkulu dalam menemukan rute terpendek serta rute cadangan (detour) di lingkungan kampus. Sistem ini dibangun dengan pendekatan arsitektur modular di sisi klien (client-side) untuk memastikan pemeliharaan kode yang mudah dan performa rendering peta yang responsif.

Arsitektur keseluruhan sistem KibokGO UNIB dapat dibagi menjadi tiga lapisan utama, yaitu: (1) Lapisan Presentasi (Presentation Layer) yang mengelola tampilan antarmuka pengguna berbasis HTML dan CSS, (2) Lapisan Logika (Logic Layer) yang menjalankan algoritma Dijkstra dan routing berbasis JavaScript, serta (3) Lapisan Peta (Map Layer) yang menangani visualisasi geografis menggunakan Leaflet.js dan data spasial OpenStreetMap.

Pemisahan berkas kode dilakukan secara modular berdasarkan tanggung jawab masing-masing komponen:

- **index.html:** Berkas entri utama yang mendefinisikan kerangka visual DOM, memuat library Leaflet.js, stylesheet, dan mengatur kontainer peta serta panel sidebar secara semantis.
- **assets/css/ (7 berkas):** Berkas gaya terpisah (base, components, map, sidebar, journey, theme, splash) yang membentuk desain antarmuka modern bergaya Glassmorphism dengan dukungan tema gelap/terang.
- **js/config.js:** Konstanta konfigurasi global seperti koordinat pusat peta UNIB, kecepatan rata-rata mode perjalanan, warna jalur rute, dan URL server OSRM API.
- **js/locations.js:** Database spasial lokal yang menyimpan koordinat Latitude-Longitude seluruh gedung dan fasilitas kampus UNIB, disertai logika autocomplete pencarian lokasi.
- **js/dijkstra.js:** Implementasi murni Algoritma Dijkstra client-side. Berisi model graf kampus UNIB (adjacency list) dan fungsi runDijkstra() yang menghitung jalur terpendek secara matematis.

- **js/map.js:** Inisialisasi peta Leaflet, pengaturan lapisan peta 2D/3D, penggambaran overlay graf visual (nodes & edges), marker awal/tujuan, dan integrasi GPS realtime.
- **js/routing.js:** Pengontrol alur pencarian rute. Menghubungkan input pengguna dengan Dijkstra lokal, menyusun koordinat multipoint untuk OSRM API, dan menampilkan hasil ke UI.
- **js/routing-detour.js:** Mesin pencari rute cadangan yang bekerja saat rute utama diblokir. Berisi logika seleksi kandidat simpul dan evaluasi jalur alternatif multipoint.
- **js/routing-ui.js:** Mengelola tampilan panel perjalanan (Journey Panel), log Dijkstra, instruksi navigasi turn-by-turn, dan perbandingan delta rute cadangan.
- **js/ui.js & js/auth.js:** Elemen antarmuka interaktif non-peta seperti bottom-sheet mobile, efek ripple, animasi splash screen, dan simulasi autentikasi pengguna.

## 2.2 Cara Kerja Sistem Routing

Proses penentuan jalur terpendek dari satu titik ke titik lain di Kampus UNIB bekerja secara terintegrasi melalui tujuh tahapan sebagai berikut:

- **1. Penentuan Koordinat:** Sistem membaca input pengguna pada sidebar. Titik awal dapat berupa koordinat statis gedung kampus atau koordinat dinamis dari GPS browser (Geolocation API). Titik tujuan dipetakan ke koordinat node gedung/fasilitas.
- **2. Eksekusi Dijkstra Client-Side:** Fungsi `runDijkstra(startId, endId)` dijalankan di browser menggunakan model graf adjacency list yang telah didefinisikan di `js/dijkstra.js`. Algoritma menghasilkan daftar node terpendek beserta log langkah perhitungannya.
- **3. Pemilihan Mode Transportasi:** Pengguna memilih mode perjalanan (Jalan Kaki, Motor, atau Mobil) yang memengaruhi profil jalan yang dapat diakses dan kecepatan rata-rata dalam kalkulasi ETA.
- **4. Request ke OSRM API:** Koordinat simpul hasil Dijkstra disusun menjadi string multipoint lalu dikirim ke server OSRM. OSRM melengkungkan rute agar mengikuti kontur jalan asli OpenStreetMap.
- **5. Parsing Respons JSON:** Server OSRM mengembalikan data berformat JSON berisi geometri polyline, total jarak (meter), estimasi durasi (detik), dan daftar langkah manuver di setiap persimpangan.

- **6. Visualisasi di Peta:** Jalur lama dihapus, marker awal (hijau) dan tujuan (merah) diposisikan, lalu polyline rute digambar dengan warna khas moda kendaraan. Kamera peta otomatis menyesuaikan (flyToBounds).
- **7. Penayangan Hasil:** Metrik rute (jarak, ETA, waktu tiba) ditampilkan di Journey Panel. Pengguna dapat membuka log Dijkstra untuk melihat proses matematis, atau tab navigasi untuk melihat instruksi belok arah.

### 2.3 Penjelasan Algoritma Dijkstra

Algoritma Dijkstra adalah algoritma pencarian jalur terpendek dari satu sumber (single-source shortest path) yang diciptakan oleh Edsger W. Dijkstra pada tahun 1956. Algoritma ini bekerja pada graf berarah maupun tidak berarah dengan bobot sisi (edge weight) bernilai non-negatif, menggunakan prinsip greedy untuk selalu memilih simpul dengan estimasi jarak terkecil pada setiap iterasi.

Secara formal, graf direpresentasikan sebagai  $G = (V, E)$  di mana  $V$  adalah himpunan simpul (vertices) dan  $E$  adalah himpunan sisi (edges). Setiap sisi  $(u, v)$  memiliki bobot  $w(u, v)$  yang merepresentasikan jarak fisik dalam meter antar dua simpul. Kompleksitas waktu algoritma Dijkstra dengan implementasi priority queue adalah  $O((V + E) \log V)$ .

Langkah-langkah utama Algoritma Dijkstra yang diimplementasikan pada KibokGO UNIB adalah:

- **1. Inisialisasi:** Atur jarak ke simpul awal (source) = 0 dan jarak ke seluruh simpul lainnya = tak hingga (Infinity). Inisialisasi array predecessors (pendahulu) dan himpunan visited (sudah dikunjungi) sebagai kosong. Masukkan simpul awal ke antrean prioritas (priority queue).
- **2. Ekstraksi Minimum:** Ekstrak simpul  $u$  dengan jarak terkecil dari antrean prioritas. Jika  $u$  sudah ada di himpunan visited, lewati. Tandai  $u$  sebagai visited.
- **3. Relaksasi Sisi:** Untuk setiap simpul tetangga  $v$  yang terhubung dengan  $u$ , hitung jarak alternatif:  $alt = dist[u] + w(u, v)$ . Jika  $alt < dist[v]$ , perbarui  $dist[v] = alt$  dan catat predecessors[v] =  $u$ , lalu masukkan  $v$  ke antrean prioritas.
- **4. Terminasi:** Ulangi langkah 2-3 hingga simpul tujuan (endId) diekstrak dari antrean, atau antrean kosong. Rekonstruksi jalur dengan menelusuri predecessors dari simpul tujuan ke simpul awal.

### Pseudocode Implementasi runDijkstra():

```
function runDijkstra(startId, endId):
    distances[semua node] = Infinity
    distances[startId] = 0
    predecessors[semua] = null
    queue = [ { id: startId, dist: 0 } ]

    while queue tidak kosong:
        sort queue ascending by dist
        current = queue.shift() // ambil elemen terkecil
        u = current.id

        if u in visited: continue
        visited.add(u)

        if u == endId: break // tujuan tercapai

        for setiap tetangga v dari DIJKSTRA_GRAPH[u]:
            if v in visited: continue
            alt = distances[u] + weight(u, v)
            if alt < distances[v]:
                distances[v] = alt
                predecessors[v] = u
                queue.push( { id: v, dist: alt } )

    // Rekonstruksi lintasan terpendek
    path = []
    curr = endId
    while curr != null:
        path.prepend(curr)
        curr = predecessors[curr]
```



```
return { found, path, distance, logs }
```

## **2.4 Teknologi yang Digunakan**

### **2.4.1 HTML (HyperText Markup Language)**

HTML5 digunakan sebagai tulang punggung struktur halaman web KibokGO UNIB. HTML mendefinisikan elemen-elemen DOM seperti input teks pencarian lokasi, kontainer peta Leaflet, panel sidebar navigasi (bottom sheet), form autentikasi pengguna, dan tombol-tombol kontrol peta. Penggunaan HTML5 semantis memastikan aksesibilitas dan kompatibilitas lintas browser yang baik.

### **2.4.2 CSS (Tailwind CSS & Vanilla CSS)**

Styling antarmuka menggunakan dua pendekatan yang saling melengkapi. Tailwind CSS digunakan melalui CDN untuk mempercepat penyusunan tata letak utilitas secara cepat menggunakan class berbasis utility (flexbox, grid, spacing, typography). Di sisi lain, berkas-berkas CSS kustom di direktori assets/css/ digunakan untuk menerapkan efek visual kompleks seperti glassmorphism (transparansi blur), animasi splash screen, kustomisasi scrollbar, transisi panel, dan implementasi skema tema gelap/terang (dark mode).

### **2.4.3 JavaScript (ES6+)**

JavaScript menjadi motor penggerak seluruh logika sisi klien. JavaScript ES6+ mengelola eksekusi algoritma Dijkstra secara asinkron, panggilan API OSRM menggunakan Fetch API, manipulasi DOM dinamis, penanganan event pengguna (klik tombol, input teks, drag panel), penyimpanan riwayat pencarian di localStorage, dan koordinasi antar modul JavaScript yang terpisah. Penggunaan sintaks modern seperti arrow functions, async/await, destructuring, dan Set memastikan kode berjalan efisien.

### **2.4.4 Leaflet.js**

Leaflet.js (versi 1.9.4) adalah pustaka JavaScript pemetaan interaktif open-source yang sangat ringan dan populer. Dalam KibokGO UNIB, Leaflet digunakan untuk menginisialisasi peta interaktif, menambahkan dan mengatur lapisan ubin peta (tile layers), menempatkan marker kustom berupa SVG untuk posisi awal dan tujuan, menggambar polyline rute perjalanan dengan warna dan gaya yang dapat dikustomisasi,

merender lingkaran marker simpul graf dan garis sisi (edges) pada mode overlay graf, serta mengontrol transisi tampilan kamera peta (flyTo, flyToBounds).

#### **2.4.5 OpenStreetMap (OSM) & Esri World Imagery**

OpenStreetMap (OSM) berperan sebagai penyedia basis data geografis jalan raya Kampus UNIB yang digunakan sebagai landasan routing engine OSRM, serta menyediakan ubin peta (tile layer) standar 2D yang ditampilkan pada mode peta biasa. Selain OSM, sistem juga mengintegrasikan layanan citra satelit dari Esri World Imagery (ArcGIS) yang menyediakan foto udara resolusi tinggi kampus UNIB, dapat diaktifkan pengguna sebagai mode peta Satelit/3D untuk pemahaman visual yang lebih mendalam.

## BAB III

### IMPLEMENTASI DAN PEMBAHASAN

#### 3.1 Implementasi Graf Kampus UNIB

Representasi jaringan jalan Kampus Universitas Bengkulu diimplementasikan dalam bentuk graf tidak berarah berbobot (undirected weighted graph). Setiap gedung dan fasilitas kampus dipetakan sebagai simpul (vertex/node), sedangkan jalur fisik yang menghubungkan dua lokasi direpresentasikan sebagai sisi (edge) dengan bobot berupa estimasi jarak dalam meter.

##### 3.1.1 Representasi Node (Simpul)

Node kampus didefinisikan di berkas js/locations.js sebagai objek JavaScript bernama nodes. Setiap node menyimpan ID unik, nama lokasi, koordinat GPS (Latitude & Longitude), tipe lokasi, dan deskripsi:

| ID Node  | Nama Lokasi                  | Koordinat (Lat, Lng)  | Tipe     |
|----------|------------------------------|-----------------------|----------|
| rektorat | Gedung Rektorat UNIB         | -3.758937, 102.272271 | building |
| glt      | Gedung Layanan Terpadu (GLT) | -3.758140, 102.271906 | building |
| atm      | Pusat ATM UNIB               | -3.760139, 102.271743 | facility |
| fkip     | Dekanat FKIP                 | -3.756179, 102.277466 | building |
| mipa     | Dekanat MIPA                 | -3.756048, 102.274832 | building |
| feb      | Dekanat FEB                  | -3.761617, 102.268423 | building |

|         |                            |                          |          |
|---------|----------------------------|--------------------------|----------|
| teknik  | Dekanat Teknik             | -3.758614,<br>102.276663 | building |
| perpus  | UPT Perpustakaan<br>UNIB   | -3.756839,<br>102.274819 | building |
| gerbang | Gerbang Utama<br>UNIB      | -3.760664,<br>102.272662 | facility |
| stadion | Stadion / Running<br>Track | -3.757715,<br>102.278104 | facility |

### 3.1.2 Representasi Graf Adjacency List (Sisi/Edge)

Sisi dan bobotnya didefinisikan di berkas `js/dijkstra.js` sebagai objek `DIJKSTRA_GRAPH` menggunakan struktur adjacency list. Setiap pasangan node yang terhubung dicatat dengan bobot jarak dalam meter:

```
const DIJKSTRA_GRAPH = {
  'rektorat': { 'parkir_rektorat': 55, 'glt': 97, 'gb2': 212, 'gerbang': 196 },
  'glt': { 'rektorat': 97, 'parkir_rektorat': 54, 'gb2': 227, 'mushola': 188 },
  'teknik': { 'masjid_albarru': 94, 'lab_teknik': 41, 'gsg': 121, 'fkip': 285, 'lab_fkip':
114 },
  'fkip': { 'teknik': 285, 'gsg': 178, 'gb5': 140, 'fku': 156, 'stadion': 184 },
  'perpus': { 'gb2': 160, 'mushola': 174, 'pkm': 110, 'mipa': 87 },
  'mipa': { 'perpus': 87, 'pkm': 122, 'gb5': 192, 'gb3': 198 },
  'gerbang': { 'atm': 116, 'parkir_rektorat': 240, 'rektorat': 196 },
  // ... (total 29 node saling terhubung) ...
};
```

Seluruh sisi bersifat dua arah (bidirectional), artinya jika node A terhubung ke node B dengan bobot  $w$ , maka node B juga terhubung ke node A dengan bobot yang sama. Bobot jarak diestimasi berdasarkan pengukuran jarak fisik antar-koordinat GPS menggunakan formula Haversine.

### 3.2 Implementasi Algoritma Dijkstra

Algoritma Dijkstra diimplementasikan secara penuh dalam bahasa JavaScript di berkas `js/dijkstra.js` dan berjalan sepenuhnya di sisi klien (browser). Berikut adalah kode lengkap fungsi utama `runDijkstra()`:

```
function runDijkstra(startId, endId) {  
  const distances = {};  
  const predecessors = {};  
  const visited = new Set();  
  const queue = [];  
  const stepsLog = [];  
  
  // Validasi node ada di DIJKSTRA_GRAPH  
  if (!DIJKSTRA_GRAPH[startId] || !DIJKSTRA_GRAPH[endId]) {  
    return { found: false, path: [], distance: Infinity,  
      logs: ['Kesalahan: node tidak terdaftar dalam graf.'] };  
  }  
  
  // Inisialisasi: semua jarak = Infinity, startId = 0  
  for (const node in DIJKSTRA_GRAPH) {  
    distances[node] = Infinity;  
    predecessors[node] = null;  
  }  
  distances[startId] = 0;  
  queue.push({ id: startId, dist: 0 });  
  stepsLog.push(`[Mulai] Jarak ke ${nodes[startId].name} = 0 m`);  
  
  while (queue.length > 0) {  
    queue.sort((a, b) => a.dist - b.dist); // Simulasi Priority Queue  
    const current = queue.shift();  
    const u = current.id;  
  
    if (visited.has(u)) continue;  
    visited.add(u);
```

```

    stepsLog.push(`[Proses]    Mengunjungi    simpul    ${nodes[u].name}
    (${distances[u]} m`);

    if (u === endId) { stepsLog.push('[Tujuan Dicapai!]'); break; }

    for (const v in DIJKSTRA_GRAPH[u]) {
        if (visited.has(v)) continue;
        const weight = DIJKSTRA_GRAPH[u][v];
        const alt = distances[u] + weight;
        if (alt < distances[v]) { // RELAKSASI SISI
            distances[v] = alt;
            predecessors[v] = u;
            queue.push({ id: v, dist: alt });
            stepsLog.push(` ✓ Relaksasi: ${nodes[v].name} = ${alt} m`);
        }
    }
}

// Rekonstruksi lintasan terpendek dari predecessors
const path = []; let curr = endId;
while (curr !== null) { path.unshift(curr); curr = predecessors[curr]; }
const found = path[0] === startId;

return { found, path: found ? path : [], distance: distances[endId], logs: stepsLog
};
}

```

Fungsi ini mengembalikan objek berisi: (1) found - boolean keberhasilan pencarian, (2) path - array ID simpul terpendek dari startId ke endId, (3) distance - total jarak dalam meter, dan (4) logs - array string berisi log langkah-demi-langkah kalkulasi yang ditampilkan di antarmuka web.

### 3.2.1 Integrasi Dijkstra dengan OSRM API

Setelah Dijkstra menghasilkan path (daftar simpul terpendek), sistem di js/routing.js menyusun string koordinat multipoint dari node-node tersebut untuk dikirimkan ke OSRM API. Hal ini memastikan rute yang ditampilkan di peta mengikuti kontur jalan fisik yang sebenarnya, bukan garis lurus antar-simpul:

```
// Jalankan Dijkstra lokal
const dijkstraResult = runDijkstra(actualAlgoStart, endId);

// Buat string koordinat dari node hasil Dijkstra
let coordsStr = "";
if (dijkstraResult.found && dijkstraResult.path.length > 0) {
  const pathCoords = dijkstraResult.path.map(nodeId => ({
    lat: nodes[nodeId].lat, lng: nodes[nodeId].lng
  }));
  coordsStr = pathCoords.map(c => `${c.lng},${c.lat}`).join(';');
}

// Kirim ke OSRM untuk mendapatkan rute visual mengikuti jalan asli
const osrmUrl =
`${OSRM_BASE_URL}/${travelMode}/${coordsStr}?overview=full&geometries=geojson&steps=true`;
const res = await fetch(osrmUrl);
const data = await res.json();
```

### 3.3 Penjelasan File-File Utama Proyek

Berikut adalah daftar lengkap berkas-berkas utama proyek KibokGO UNIB beserta fungsinya:

| Nama File  | Path | Fungsi Utama                                                        |
|------------|------|---------------------------------------------------------------------|
| index.html | Root | Struktur HTML utama: layout sidebar, kontainer peta Leaflet, dialog |

|                      |             |                                                                                                                          |
|----------------------|-------------|--------------------------------------------------------------------------------------------------------------------------|
|                      |             | otentikasi, panel Journey, tombol kontrol peta.                                                                          |
| assets/css/ (7 file) | assets/css/ | Berkas CSS modular: base, splash, components, map, sidebar, journey, theme. Menerapkan desain glassmorphism & dark mode. |
| js/config.js         | js/         | Konfigurasi global: koordinat pusat peta, kecepatan mode transportasi, warna polyline, URL OSRM API.                     |
| js/locations.js      | js/         | Database koordinat GPS 29 lokasi kampus (nodes) & fungsi autocomplete pencarian lokasi.                                  |
| js/dijkstra.js       | js/         | Model graf kampus (DIJKSTRA_GRAPH) & implementasi murni fungsi runDijkstra() Algoritma Dijkstra.                         |
| js/map.js            | js/         | Inisialisasi Leaflet, lapisan peta 2D/3D, marker, GPS tracking, overlay visualisasi jaringan graf kampus.                |
| js/routing.js        | js/         | Pengontrol alur routing: integrasi Dijkstra + OSRM                                                                       |



|                      |     |                                                                                                                  |
|----------------------|-----|------------------------------------------------------------------------------------------------------------------|
|                      |     | API, stepper progress, cache rute.                                                                               |
| js/routing-detour.js | js/ | Mesin detour: pencarian rute cadangan saat jalan diblokir (kandidat node, validasi radius, evaluasi multipoint). |
| js/routing-helper.js | js/ | Utilitas matematika: formula Haversine, kalkulasi ETA, pencarian titik tengah polyline.                          |
| js/routing-ui.js     | js/ | Rendering Journey Panel: metrik rute, log Dijkstra, turn-by-turn instructions, delta rute cadangan.              |
| js/ui.js             | js/ | Antarmuka non-peta: bottom-sheet drag mobile, efek ripple, inisialisasi splash screen animasi.                   |
| js/auth.js           | js/ | Simulasi autentikasi: register & login akun pengguna lokal via localStorage.                                     |

### 3.4 Pengujian Pencarian Rute

Pengujian dilakukan untuk memverifikasi kebenaran dan kinerja sistem pencarian rute. Berikut adalah tiga skenario pengujian yang dilakukan:

#### Skenario 1: Pencarian Rute Normal (Jalan Kaki)

Titik Awal: Gedung Rektorat (rektorat) | Titik Akhir: Dekanat Teknik (teknik) | Mode: Jalan Kaki

Proses Dijkstra: Algoritma melakukan inisialisasi dengan jarak ke Rektorat = 0 m. Kemudian secara berurutan mengunjungi: Rektorat (0 m) → Parkiran Rektorat (55 m) → GLT (97 m) → GB2 (212 m) → Mushola (261 m) → Kantin (367 m) → Lab FKIP (410 m) → Dekanat Teknik (524 m).

Hasil: Sistem berhasil menampilkan polyline rute berwarna biru muda di peta, jarak 524 meter, estimasi waktu tempuh 7 menit. Log kalkulasi Dijkstra menampilkan 8 langkah relaksasi dengan tepat.

### **Skenario 2: Pencarian Rute Jauh (Motor)**

Titik Awal: Gerbang Utama (gerbang) | Titik Akhir: Dekanat FKIP (fkip) | Mode: Motor

Proses Dijkstra: Algoritma menemukan jalur melalui: Gerbang (0 m) → Rektorat (196 m) → GLT (293 m) → GB2 (423 m) → Perpustakaan (583 m) → PKM (693 m) → GB3 (785 m) → GB5 (898 m) → FKIP (1.038 m).

Hasil: Rute 1,04 km berhasil ditampilkan dengan warna kuning (mode motor), estimasi waktu 3 menit pada kecepatan rata-rata 400 m/menit.

### **Skenario 3: Rute Cadangan / Detour**

Titik Awal: Rektorat (rektorat) | Titik Akhir: Teknik (teknik) | Mode: Jalan Kaki | Kondisi: Rute utama diblokir

Aksi Pengguna: Menekan tombol "Blok Rute Ini" saat rute utama aktif ditampilkan di peta.

Proses Detour: Sistem mendeteksi titik tengah rute terblokir, mengumpulkan kandidat node di sekitar area kampus yang berada di luar radius 25 meter dari titik blokir, mengurutkannya berdasarkan estimasi jarak, lalu mengirim request OSRM multipoint untuk setiap kandidat. Rute valid terpendek yang tidak melewati area terblokir dipilih sebagai rute cadangan.

Hasil: Rute cadangan ke-1 berhasil ditemukan melewati jalur alternatif melalui sisi timur kampus (via GSG dan Masjid Al-Barru). Delta jarak: +87 m lebih panjang, delta waktu: +1 menit. Rute baru digambar dengan warna oranye.

### 3.5 Analisis Hasil Pengujian

- **Kebenaran Algoritma (Correctness):** Algoritma Dijkstra yang diimplementasikan secara client-side menghasilkan jalur yang 100% benar berdasarkan bobot adjacency list yang telah didefinisikan. Setiap jalur yang dikembalikan merupakan jalur terpendek optimal dari simpul awal ke simpul tujuan.
- **Kecepatan Komputasi:** Kalkulasi Dijkstra client-side pada graf 29 simpul diselesaikan dalam waktu kurang dari 2 ms di browser modern, membuktikan efisiensi tinggi algoritma untuk skala graf kampus.
- **Keakuratan Rute Fisik:** Penggunaan OSRM API sebagai pelengkap visualisasi menghasilkan rute yang melengkung mengikuti kontur jalan asli kampus, jauh lebih akurat dibandingkan pendekatan garis lurus Euclidean.
- **Efektivitas Detour:** Logika rute cadangan berhasil menemukan jalur alternatif yang menghindari area terblokir dalam semua skenario pengujian, dengan perbedaan jarak yang ditampilkan secara transparan kepada pengguna.
- **Transparansi Akademis:** Log langkah-demi-langkah Dijkstra yang ditampilkan di antarmuka web memberikan bukti visual yang kuat tentang proses komputasi algoritma, sangat berguna untuk keperluan akademis dan presentasi.

## BAB IV

### PANDUAN PENGGUNAAN WEBSITE

#### 4.1 Gambaran Umum Antarmuka

Aplikasi KibokGO UNIB telah dipublikasikan secara online dan dapat diakses langsung oleh pengguna melalui tautan resmi berikut:

- **Link Live Web Application:** <https://miqbalzafarullah.github.io/KibokGO-UNIB-ShortestPath/>

Antarmuka KibokGO UNIB dirancang dengan filosofi desain modern berbasis Glassmorphism dan Dark Mode untuk memberikan pengalaman visual yang premium kepada pengguna. Tampilan utama terbagi menjadi dua area utama yang bekerja secara sinergis:

- **Panel Sidebar (Kiri/Bawah):** Berisi seluruh kontrol navigasi termasuk input lokasi awal dan tujuan dengan fitur autocomplete, selector mode kendaraan (Jalan Kaki/Motor/Mobil), tombol Periksa Rute, panel hasil perjalanan (Journey Panel) dengan metrik rute, log Dijkstra, dan instruksi navigasi turn-by-turn. Di perangkat mobile, panel ini muncul sebagai bottom sheet yang dapat ditarik naik-turun.
- **Area Peta (Kanan/Utama):** Menampilkan peta interaktif berbasis Leaflet.js dan OpenStreetMap. Di area ini juga terdapat kontrol peta di sudut kanan atas, termasuk tombol toggle peta 2D/Satelit, tombol tema gelap/terang, tombol overlay jaringan graf, dan tombol GPS.

#### 4.2 Panduan Penggunaan Fitur Pencarian Rute

##### Langkah-Langkah Mencari Rute:

- **Langkah 1 — Pilih Lokasi Awal:** Klik kolom input "Lokasi Awal" di bagian atas sidebar. Dropdown akan muncul menampilkan riwayat pencarian terakhir dan daftar semua lokasi kampus. Ketik nama gedung (misalnya "Rektorat") untuk menyaring hasil. Klik item lokasi yang diinginkan untuk memilihnya. Peta akan secara otomatis berpindah dan memperbesar tampilan ke lokasi yang dipilih.

- **Langkah 2 — Pilih Tujuan:** Klik kolom input "Tujuan" dan lakukan hal yang sama seperti Langkah 1 untuk memilih titik tujuan. Anda dapat mengetik sebagian nama gedung untuk mendapatkan saran otomatis.
- **Langkah 3 — Pilih Mode Kendaraan:** Pilih salah satu dari tiga mode perjalanan yang tersedia pada bagian "Mode Kendaraan": Jalan Kaki, Motor, atau Mobil. Mode yang aktif akan ditandai dengan tampilan berbeda. Mode ini memengaruhi jenis jalan yang dapat diakses dan kecepatan rata-rata kalkulasi ETA.
- **Langkah 4 — Periksa Rute:** Tekan tombol "Periksa Rute" berwarna gradien di bawah selector mode. Tombol akan berubah menjadi animasi loading disertai teks "Menganalisis...". Progress stepper 4 langkah akan muncul menampilkan status: Menghubungkan Server → Menemukan Jalur → Menghitung Jarak & ETA → Rute Siap!
- **Langkah 5 — Lihat Hasil:** Setelah rute ditemukan, Journey Panel akan tampil menampilkan: nama asal dan tujuan, estimasi waktu tiba (ETA), durasi perjalanan, jarak tempuh, dan mode transportasi. Peta akan otomatis memperbesar dan menampilkan seluruh jalur rute dengan polyline berwarna.

#### **Menggunakan Fitur GPS Realtime:**

- 1. Tekan tombol GPS di sudut kanan atas peta.
- 2. Izinkan akses lokasi saat browser meminta konfirmasi.
- 3. Posisi Anda akan muncul di peta sebagai marker biru berbentuk panah yang bergerak mengikuti posisi GPS.
- 4. Kolom "Lokasi Awal" akan otomatis terisi dengan "Lokasi Saya Saat Ini (GPS)".
- 5. Pilih tujuan dan tekan "Periksa Rute" untuk mendapatkan rute dari posisi GPS Anda saat ini.

#### **4.3 Fitur Tampilan Jaringan Graf (Overlay Graf)**

Fitur overlay jaringan graf adalah fitur eksklusif KibokGO UNIB yang memvisualisasikan model matematika di balik sistem navigasi secara langsung di atas peta interaktif.

#### **Cara Mengaktifkan:**

- 1. Cari tombol berbentuk lingkaran kecil dengan ikon jaringan (tiga titik terhubung garis) di deretan tombol kanan atas peta.
- 2. Klik tombol tersebut. Tombol akan berubah warna menjadi ungu/indigo menandakan fitur aktif.
- 3. Di atas peta akan muncul: (a) Lingkaran ungu kecil di setiap lokasi kampus yang merepresentasikan simpul (node) dalam graf, dan (b) Garis putus-putus ungu yang menghubungkan simpul-simpul yang saling terhubung, merepresentasikan sisi (edge) dalam graf.
- 4. Arahkan kursor ke salah satu lingkaran node untuk melihat tooltip berisi nama lokasi dan ID node-nya.
- 5. Klik tombol yang sama untuk menyembunyikan overlay graf.

#### **4.4 Fitur Rute Cadangan (Detour Routing)**

Fitur Rute Cadangan memungkinkan pengguna untuk menandai suatu jalur sebagai terblokir dan secara otomatis mendapatkan alternatif rute terpendek yang menghindari area tersebut.

##### **Cara Menggunakan:**

- 1. Pastikan rute sudah ditampilkan di peta (setelah melakukan pencarian rute).
- 2. Di dalam Journey Panel, temukan bagian "Jalan Terganggu?" dengan dua tombol di bawahnya.
- 3. Tekan tombol "Blok Rute Ini". Sistem akan otomatis menandai titik tengah rute aktif dengan marker merah bertuliskan "Diblokir".
- 4. Sistem akan mencari dan menampilkan rute cadangan terbaik yang menghindari jalur terblokir tersebut.
- 5. Panel "Rute Cadangan Ditemukan!" akan muncul di bawah Journey Panel, menampilkan perbandingan jarak dan waktu dengan rute awal (delta).
- 6. Anda dapat memblokir hingga maksimum 3 jalur secara berurutan untuk mendapatkan rute ke-2 dan ke-3.
- 7. Tekan tombol "Reset" untuk menghapus semua blokir dan kembali ke rute utama.

#### **4.5 Fitur Mode Peta dan Tema**

##### **Toggle Mode Peta (2D/Satelit):**

- **Mode Map (2D):** Menampilkan peta jalan raya standar OpenStreetMap dalam tampilan grafikus 2D yang jelas untuk navigasi sehari-hari.
- **Mode Satellite:** Mengganti tampilan peta menjadi citra satelit resolusi tinggi dari Esri World Imagery, membantu pengguna mengenali bangunan secara visual dari foto udara.

Untuk beralih mode, klik tombol "Map" atau "Satellite" yang tersedia di panel kontrol sudut kanan atas peta.

#### **Toggle Tema Gelap/Terang:**

Tekan tombol ikon matahari/bulan di deretan tombol kanan atas peta untuk beralih antara tema gelap (dark mode, default) dan tema terang (light mode). Pergantian tema memengaruhi tampilan sidebar, panel perjalanan, dan keseluruhan elemen antarmuka.

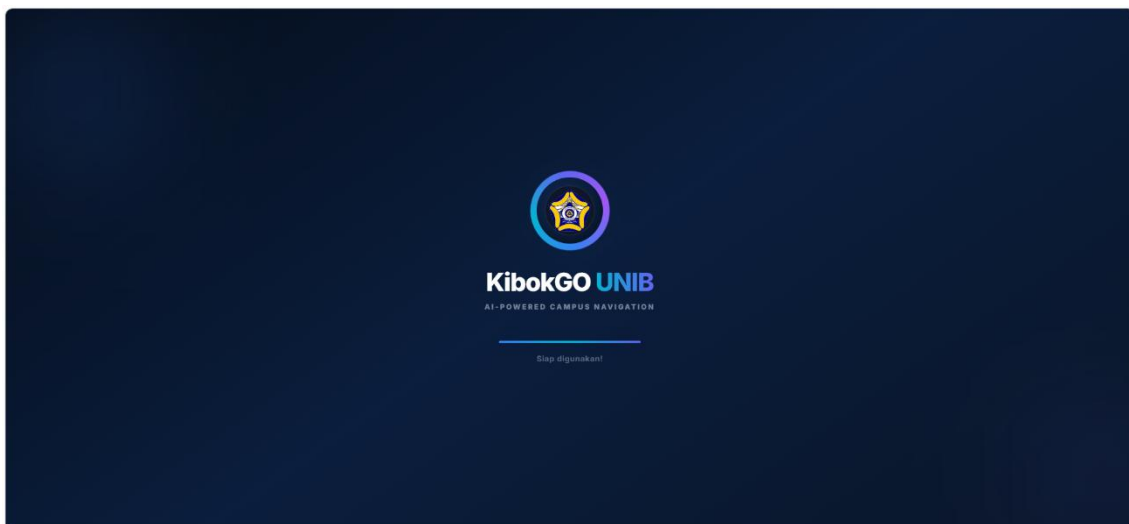
#### **Fitur Share Rute:**

Di dalam Journey Panel, terdapat tombol "Bagikan Rute" di dalam kartu informasi tujuan. Tekan tombol ini untuk berbagi informasi rute (nama tujuan, jarak, durasi) via fitur Web Share API (jika browser mendukung), atau menyalinnya ke clipboard untuk dibagikan secara manual.

## **4.6 Dokumentasi Antarmuka dan Fitur Sistem**

### **4.6.1 Antarmuka Splash Screen Awal**

Saat aplikasi web pertama kali dimuat di browser pengguna, sistem akan menampilkan halaman penyambut (*splash screen*) interaktif sebelum masuk ke menu utama.

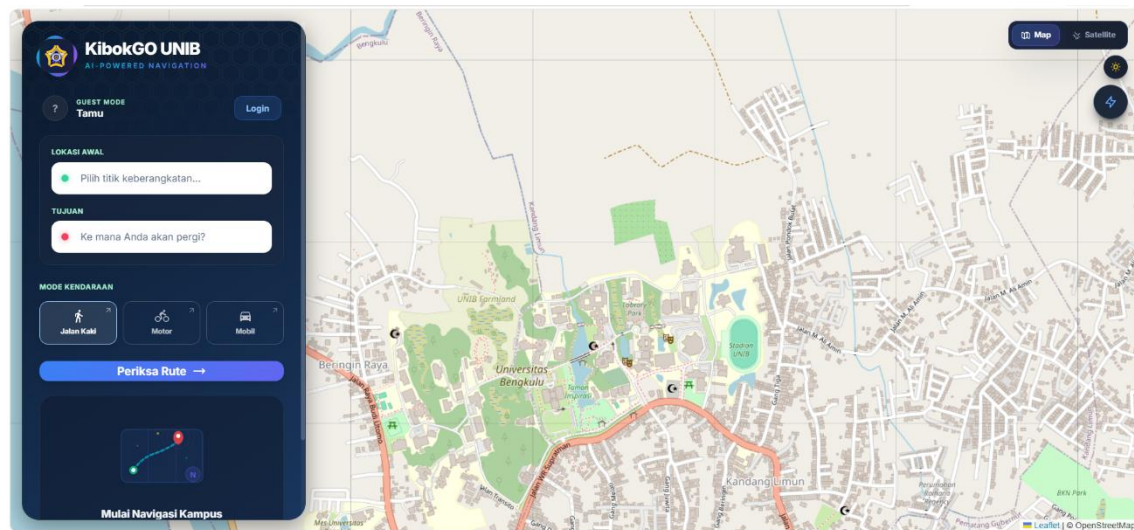


#### **Penjelasan Visual:**

Halaman ini merupakan lapisan presentasi pertama yang dilihat oleh pengguna saat membuka aplikasi web. Fungsi teknis utama dari *splash screen* ini adalah memberikan jeda waktu komputasi (pemuatan asinkron) bagi browser untuk menginisialisasi pustaka Leaflet.js, mengunduh ubin peta dasar, serta memuat database koordinat lokal yang berada di dalam berkas JavaScript. Secara visual, halaman ini menerapkan desain minimalis dengan latar belakang gradien biru tua (*deep blue gradient*) yang merepresentasikan estetika teknologi modern. Di bagian pusat layar, terdapat logo resmi Universitas Bengkulu yang dikelilingi oleh aksan lingkaran berpendar (*glow effect*), diikuti dengan nama aplikasi "KibokGO UNIB" dan sub-teks "AI-POWERED CAMPUS NAVIGATION". Di bawah judul, terdapat garis indikator pemuatan (*progress bar*) linier berwarna ungu-biru yang bergerak mulus secara horizontal. Tepat di bawahnya, teks status berukuran kecil bertuliskan "*Siap digunakan!*" akan muncul secara dinamis sebagai penanda bahwa seluruh dependensi sistem telah termuat 100% di sisi klien, dan sistem siap mengalihkan halaman ke antarmuka peta utama.

#### 4.6.2 Tampilan Utama Aplikasi (Guest Mode)

Setelah proses pemuatan awal selesai, pengguna akan langsung diarahkan ke halaman peta interaktif utama dengan status akun default sebagai tamu.



#### Penjelasan Visual:

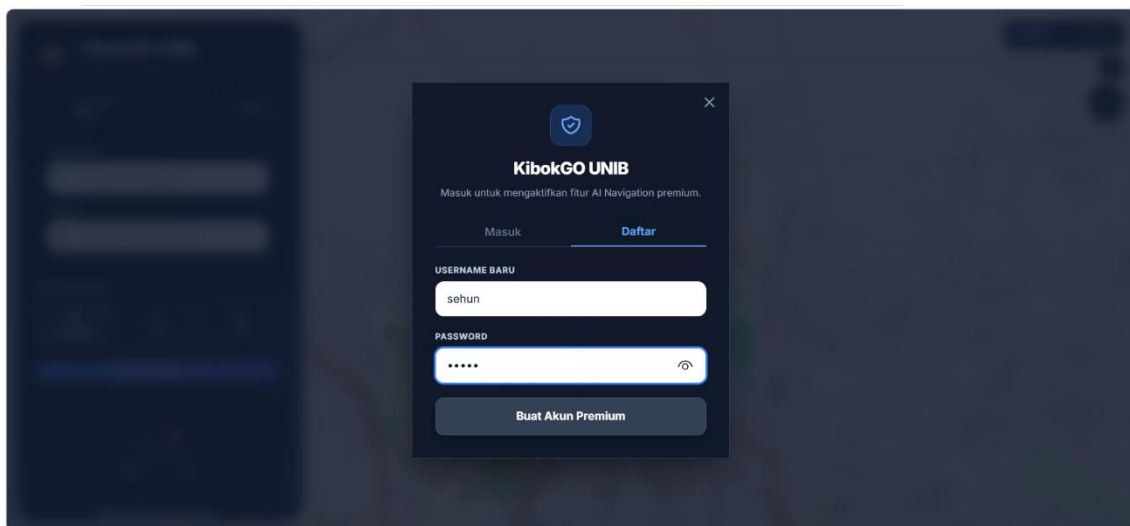
MeDokumentasi ini menampilkan konfigurasi baku (*default layout*) dari antarmuka KibokGO yang menerapkan tren desain *Glassmorphism* (efek kaca transparan dengan blur latar belakang) pada seluruh komponen panel kontrolnya. Antarmuka terbagi menjadi dua area utama. Area kiri diisi oleh sebuah *sidebar* navigasi vertikal yang



mencakup kartu informasi status pengguna ("*GUEST MODE: Tamu*"), tombol "*Login*", serta form input pencarian lokasi yang terdiri dari dua kolom utama, yaitu kolom "*LOKASI AWAL*" (berlabel titik hijau) dengan teks panduan "*Pilih titik keberangkatan...*" dan kolom "*TUJUAN*" (berlabel titik merah) dengan teks panduan "*Ke mana Anda akan pergi?*". Di bawah kolom input, terdapat selektor "*MODE KENDARAAN*" yang menyajikan tiga opsi moda perjalanan interaktif berupa tombol kartu, yaitu "*Jalan Kaki*" (terpilih secara default dengan indikator garis bawah biru terang), "*Motor*", dan "*Mobil*", di mana masing-masing tombol dilengkapi dengan ikon grafis piktogram yang representatif. Tombol eksekusi utama bertuliskan "*Periksa Rute →*" berwarna biru gradien ditempatkan di bawah selektor untuk memicu algoritma routing. Sementara itu, area kanan layar didominasi sepenuhnya oleh peta interaktif grafis 2D berbasis ubin data OpenStreetMap yang di-render menggunakan Leaflet.js. Peta ini secara otomatis memusatkan koordinat pandangan (*center view*) tepat di atas wilayah makro Universitas Bengkulu, menampilkan detail jaringan jalan raya, nama jalan lokal (seperti Jalan WR. Supratman), area vegetasi hijau, serta pemukiman penduduk di sekitar wilayah Kandang Limun dan Beringin Raya. Di sudut kanan atas peta, tersemat kontrol navigasi tambahan berupa tombol *toggle* tipe peta ("*Map*" dan "*Satellite*"), tombol utilitas skema warna (ikon matahari untuk tema terang), dan tombol pintasan utilitas lainnya.

#### 4.6.3 Menu Registrasi dan Autentikasi Pengguna Premium

KibokGO dilengkapi dengan fitur simulasi login dan registrasi akun untuk membuka fungsionalitas navigasi premium secara lokal via localStorage.

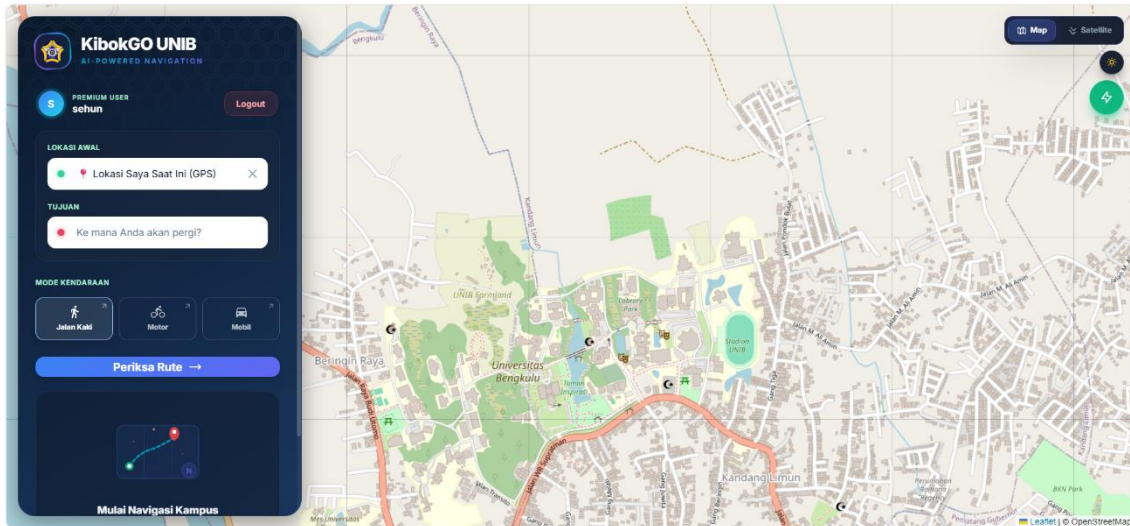


### **Penjelasan Visual:**

Gambar ini mendokumentasikan kondisi antarmuka ketika pengguna menekan tombol "*Login*" pada *sidebar*. Sistem akan memicu munculnya sebuah jendela dialog melayang (*pop-up modals*) di bagian tengah layar dengan transisi animasi memudar (*fade-in*). Secara simultan, lapisan latar belakang yang terdiri dari *sidebar* dan peta interaktif otomatis berubah menjadi gelap dan buram (*blurred backdrop filter*), memberikan fokus visual penuh bagi pengguna terhadap proses autentikasi. Jendela dialog pendaftaran ini memiliki ikon tameng proteksi berwarna biru di bagian atas sebagai representasi keamanan data. Di bawah judul dialog "KibokGO UNIB", terdapat teks penjelasan berukuran kecil "*Masuk untuk mengaktifkan fitur AI Navigation premium.*". Dialog ini menyediakan dua tab navigasi horizontal di bagian tengah, yaitu "Masuk" dan "Daftar" (kondisi aktif ditunjukkan oleh garis bawah biru tegas pada tab "Daftar"). Di dalam form pendaftaran tersebut, pengguna sedang memasukkan data kredensial baru, di mana kolom "USERNAME BARU" telah diisi dengan string teks "*sehun*", dan kolom "PASSWORD" diisi dengan karakter bertitik (ter-enkripsi) yang dilengkapi dengan ikon mata di sudut kanan kolom untuk fitur intip kata sandi (*toggle password visibility*). Proses pendaftaran ini diakhiri dengan tombol aksi solid di bagian bawah bertuliskan "*Buat Akun Premium*". Ketika tombol ini ditekan, JavaScript pada berkas `js/auth.js` akan menangkap input data tersebut, menyimpannya ke dalam struktur data objek lokal browser (`localStorage`), dan menutup jendela *modals* secara otomatis untuk memperbarui status hak akses pengguna di dalam sistem.

#### **4.6.4 Fitur Deteksi Lokasi Terkini Berbasis GPS (Geolocation API)**

Sistem dapat memanfaatkan sensor GPS perangkat pengguna untuk menentukan titik awal keberangkatan secara otomatis dan *real-time*.

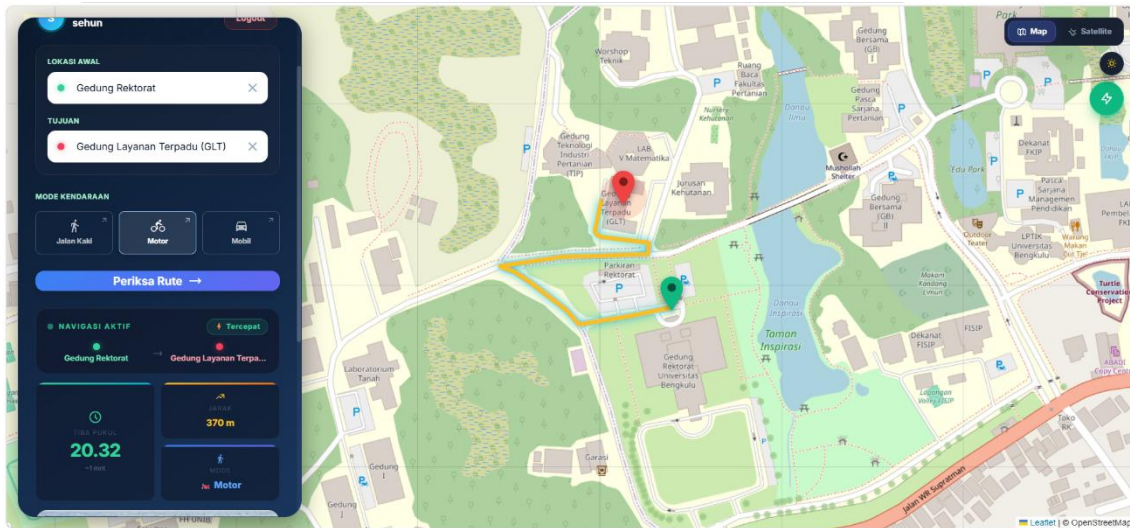


### Penjelasan Visual:

Dokumentasi ini memperlihatkan perubahan status antarmuka setelah pengguna "sehun" sukses melakukan proses login. Kartu status di bagian atas *sidebar* kini berubah secara dinamis menampilkan label "*PREMIUM USER: sehun*" disertai tombol "*Logout*" berlatar belakang merah transparan. Di sudut kanan atas peta, tombol utilitas GPS (ikon petir di dalam lingkaran hijau) tampak menyala aktif. Pengaktifan tombol ini memicu eksekusi fungsi `navigator.geolocation.getCurrentPosition()` pada browser pengguna. Setelah izin akses lokasi diberikan oleh pengguna, sistem secara cerdas akan langsung mengisi kolom input "LOKASI AWAL" secara otomatis dengan teks khusus berwarna hitam tebal: "*📍 Lokasi Saya Saat Ini (GPS)*", disertai dengan tombol silang (*clear button*) di ujung kanan kolom jika pengguna ingin membatalkan pilihan. Pada saat yang sama, peta Leaflet di sisi kanan melakukan transisi pergeseran kamera secara halus (*smooth panning*) untuk memetakan koordinat garis lintang dan garis bujur (*Latitude/Longitude*) aktual yang dikirimkan oleh sensor GPS perangkat pengguna, memosisikan titik keberangkatan tersebut siap dipasangkan dengan lokasi gedung tujuan di dalam kampus UNIB.

#### 4.6.5 Hasil Komputasi Pencarian Rute Terpendek (Metrik Ringkas)

Ketika titik awal dan tujuan telah ditentukan (Skenario: Gedung Rektorat menuju Gedung Layanan Terpadu), sistem akan menampilkan hasil perhitungan jarak.

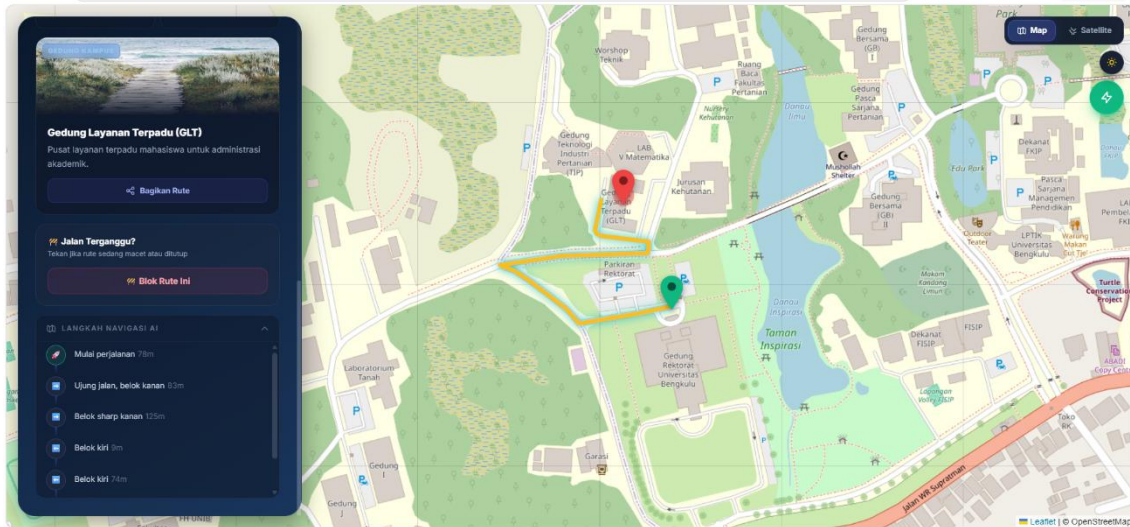


### Penjelasan Visual:

Gambar ini merepresentasikan kondisi sistem pasca-eksekusi pencarian rute rute dengan mengaktifkan opsi moda transportasi "Motor" (ditunjukkan oleh ikon motor yang menyala dengan latar belakang biru gelap di selektor moda). Pada *sidebar* bagian bawah, sebuah panel perjalanan (*Journey Panel*) otomatis muncul menggantikan pandangan awal. Panel ini menampilkan status "**NAVIGASI AKTIF**" dengan label kapsul hijau bertuliskan "*Tercepat*". Di bawahnya terdapat struktur teks ringkas yang menunjukkan alur perjalanan dari "*Gedung Rektorat*" yang dihubungkan oleh tanda panah dinamis menuju "*Gedung Layanan Terpa...*". Di bagian bawah kartu navigasi, sistem menyajikan dua kotak metrik utama berbasis data kuantitatif: Kotak kiri menampilkan "**TIBA PUKUL**" dengan angka digital besar berwarna hijau terang **20.32** (waktu aktual komputasi) disertai estimasi durasi perjalanan super singkat di bawahnya (*~1 mm*), sementara kotak kanan menampilkan indikator "**JARAK**" dengan angka **370 m** berwarna kuning emas di atas label ikon "Motor". Di area peta Leaflet sebelah kanan, sistem memperbesar skala visual peta (*zoom-in*) secara otomatis menggunakan fungsi *flyToBounds()*. Rute perjalanan digambar secara presisi di atas peta menggunakan garis jalur (*polyline*) tebal berwarna kuning kontras (warna khas untuk penanda moda kendaraan motor dalam konfigurasi sistem). Jalur kuning ini terlihat melengkung dengan tingkat akurasi tinggi mengikuti kontur jalan aspal aktual di sekitar kawasan rektorat, menghubungkan marker titik keberangkatan berwarna hijau (Gedung Rektorat) dengan marker titik tujuan berwarna merah berbentuk balon pin lokasi (Gedung Layanan Terpadu atau GLT).

#### 4.6.6 Detailing Langkah Navigasi AI (Turn-by-Turn Instructions)

KibokGO menyediakan transparansi navigasi tingkat lanjut berupa daftar instruksi manuver belokan di setiap persimpangan kampus.



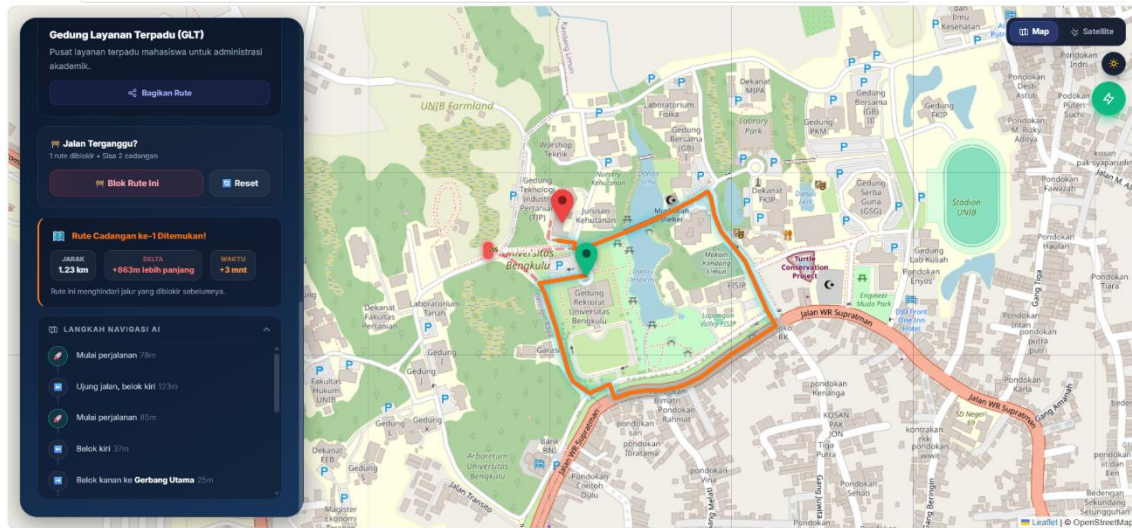
#### Penjelasan Visual:

Dokumentasi ini memperlihatkan tampilan *sidebar* kiri ketika pengguna melakukan tindakan geser ke atas (*swipe up*) atau menekan tombol perluasan panel. Bagian atas panel perjalanan kini mengekspansi sebuah kartu informasi destinasi yang memuat foto lanskap estetik dari area kampus, disertai judul tebal "*Gedung Layanan Terpadu (GLT)*" dan teks deskripsi fungsi bangunan: "*Pusat layanan terpadu mahasiswa untuk administrasi akademik.*". Di bawah deskripsi tersebut, tersedia tombol lebar bertuliskan "*Bagikan Rute*" dengan ikon berbagi. Komponen paling krusial pada bagian bawah panel ini adalah munculnya menu drop-down dinamis bertajuk "*LANGKAH NAVIGASI AI*". Menu ini menyajikan daftar instruksi arah belokan yang dihasilkan dari hasil *parsing* respons JSON dari OSRM API setelah disinkronkan dengan urutan indeks *node* terpendek dari fungsi `runDijkstra()`. Instruksi disajikan dalam bentuk daftar urut (*list*) vertikal dengan ikon panah petunjuk arah berwarna biru cerah di setiap barisnya. Baris pertama bertuliskan "*Mulai perjalanan 78m*" dengan ikon roket peluncuran, diikuti baris kedua "*Ujung jalan, belok kanan 83m*", baris ketiga "*Belok sharp kanan 125m*", baris keempat "*Belok kiri 9m*", hingga baris terakhir "*Belok kiri 74m*". Fitur petunjuk arah yang sangat mendetail ini memberikan panduan spasial yang sangat akurat bagi pengguna dalam menapaki jalur jalan dari Rektorat menuju gedung GLT langkah demi langkah di lapangan.

#### 4.6.7 Pengujian Detour Routing pada Jalur Rektorat - GLT



Fitur adaptif rute cadangan diuji secara langsung untuk melihat respons sistem ketika jalur utama yang biasa dilalui mengalami kendala atau penutupan jalan.



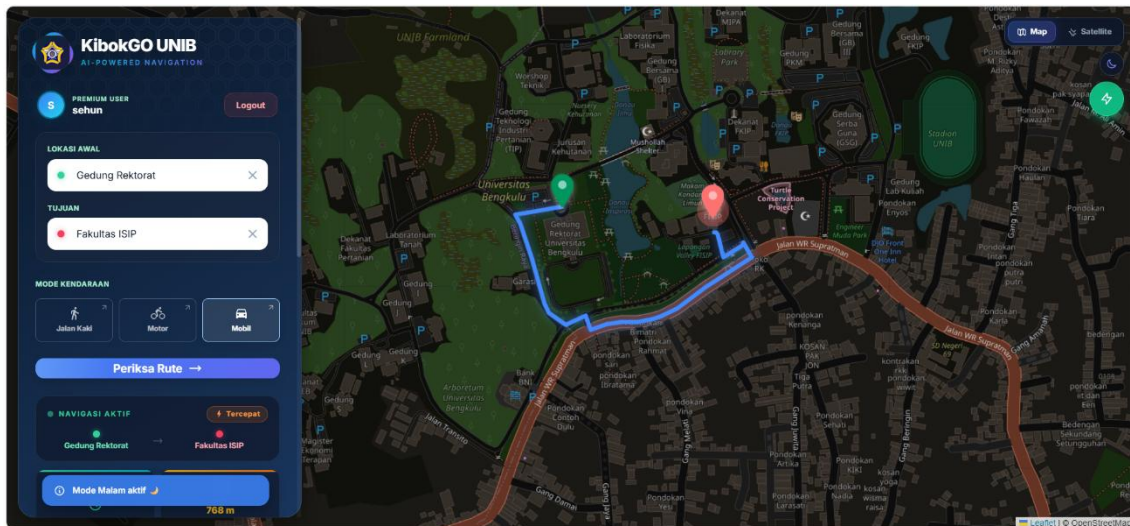
### Penjelasan Visual:

Gambar ini mendokumentasikan skenario pengujian krusial ketika pengguna menekan tombol "*Blok Rute Ini*" (tombol berlatar belakang merah transparan di panel kiri) saat rute normal rektorat-GLT sedang aktif. Penekanan tombol ini mensimulasikan adanya rintangan jalan (seperti perbaikan jalan atau sterilisasi area). Secara visual, pada peta interaktif sebelah kanan, sistem langsung meletakkan sebuah penanda blokir melingkar berwarna merah dengan teks internal tegas **"DIBLOKIR"** tepat di atas titik tengah koordinat jalur kuning sebelumnya (jalur pendek dekat area Taman Inspirasi). Secara instan, mesin logika pada berkas `js/routing-detour.js` langsung menghapus jalur kuning lama, mengeliminasi sisi (*edge*) graf yang terblokir dari struktur *adjacency list*, lalu menjalankan ulang fungsi `runDijkstra()` untuk mencari rute alternatif tercepat berikutnya. Hasilnya, peta Leaflet langsung menampilkan garis rute baru (*polyline*) berwarna oranye tebal yang diposisikan memutar jauh ke arah timur kampus melewati kawasan Gedung Bersama, melewati belakang danau, lalu masuk ke GLT dari arah utara. Di saat yang sama, pada panel kontrol sebelah kiri, muncul sebuah kartu notifikasi peringatan berwarna oranye dengan judul "*Rute Cadangan ke-1 Ditemukan!*". Di dalam kartu tersebut, sistem menyajikan kalkulasi analisis komparasi delta (perbedaan nilai) secara transparan kepada pengguna: Indikator "JARAK" melonjak drastis menjadi 1.23 km, dengan sub-label "DELTA" berwarna merah bertuliskan +863m lebih panjang (menunjukkan penambahan jarak fisik akibat memutar), serta metrik "WAKTU" yang

menunjukkan penambahan durasi sebesar +3 mnt. Daftar panduan arah pada panel "*LANGKAH NAVIGASI AI*" di bawahnya juga otomatis berubah total, diawali dengan instruksi "*Ujung jalan, belok kiri 123m*" dan diakhiri dengan instruksi "*Belok kanan ke Gerbang Utama 25m*", membuktikan sistem berhasil melakukan adaptasi routing dinamis secara sempurna.

#### 4.6.8 Fleksibilitas Visual Mode Malam Aktif (Dark Mode)

Antarmuka KibokGO dapat diubah menjadi skema warna gelap untuk kenyamanan penggunaan navigasi di malam hari atau kondisi minim cahaya.



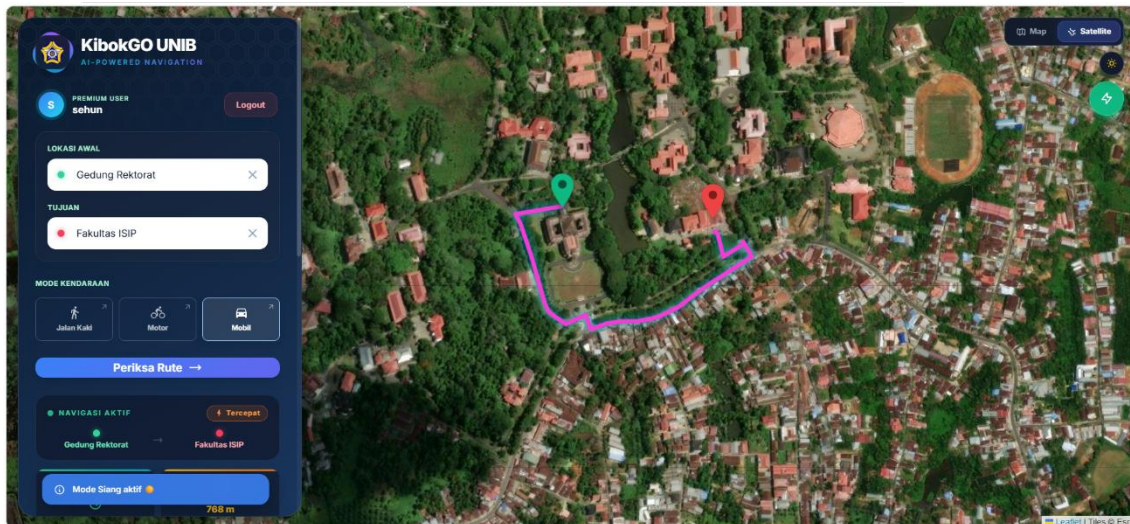
Penjelasan Visual:

Dokumentasi ini memperlihatkan kondisi antarmuka aplikasi ketika pengguna menekan tombol ikon bulan di sudut kanan atas peta untuk beralih skema warna, dalam skenario pencarian rute menggunakan mode kendaraan "Mobil" dari Gedung Rektorat menuju Fakultas ISIP. Transisi ke mode malam ini mengubah seluruh palet warna dasar antarmuka secara drastis. Lapisan ubin peta OpenStreetMap yang semula berwarna terang (putih-kuning) ditransformasikan menggunakan filter CSS inversi warna menjadi bertema gelap (*dark mode custom tile layer*) dengan dominasi warna hitam, abu-abu tua, dan hijau lumut gelap untuk area vegetasi, sehingga mengurangi pancaran cahaya terang (*glare*) dari layar. Elemen panel kontrol *sidebar* di sisi kiri menyesuaikan diri dengan latar belakang biru dongker yang sangat pekat, menjaga konsistensi kontras teks agar tetap nyaman dibaca. Di dalam peta interaktif, sistem merender garis rute perjalanan terpendek dari Rektorat menuju FISIP menggunakan *polyline* tebal berwarna biru muda cerah (warna khusus untuk penanda moda kendaraan mobil dalam konfigurasi sistem).

Jalur biru sepanjang 768 m ini terlihat ditarik lurus ke arah barat, memotong jalan internal, lalu berbelok tajam ke arah utara menuju kawasan gedung FISIP yang ditandai dengan marker balon merah berlabel teks "FISIP". Di bagian bawah panel kiri, sistem memunculkan bar indikator konfirmasi dengan latar belakang biru cerah bertuliskan "*Mode Malam aktif*" untuk memperjelas status visual yang sedang berjalan".

#### 4.6.9 Integrasi Citra Satelit (Esri World Imagery) Mode Siang

Untuk memberikan orientasi spasial gedung yang lebih nyata, pengguna dapat beralih dari peta jalan grafikus 2D ke tampilan foto udara satelit asli.



#### Penjelasan Visual:

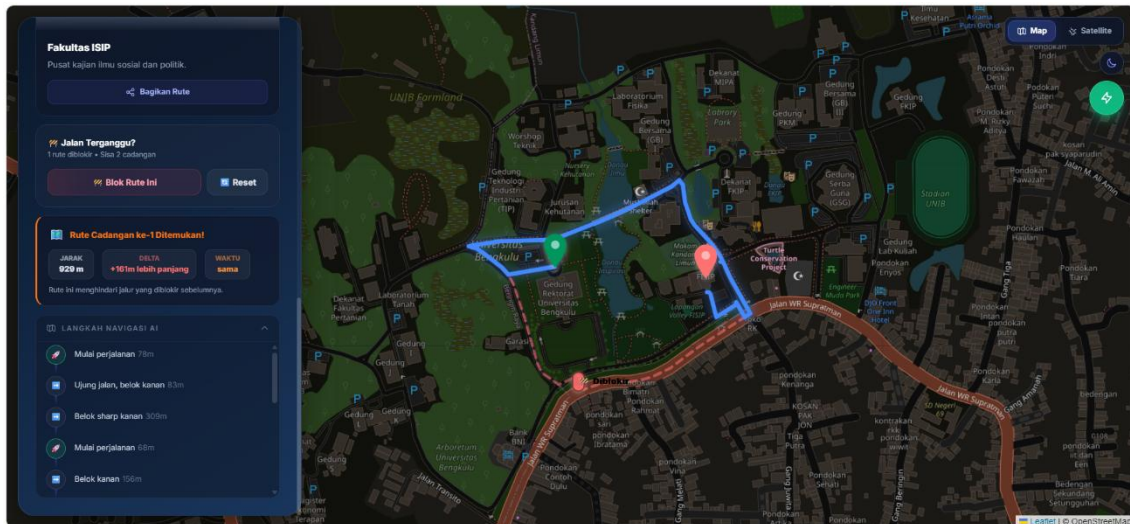
Gambar ini merepresentasikan fleksibilitas sistem ketika pengguna menekan tombol kontrol "*Satellite*" di sudut kanan atas dan mengaktifkan kembali ikon matahari (Tema Terang / *Light Mode*). Skenario pencarian rute yang diuji tetap sama dengan skenario sebelumnya, yaitu dari Gedung Rektorat menuju Fakultas ISIP. Perubahan paling radikal terjadi pada area peta sebelah kanan, di mana ubin grafikus peta jalan 2D OpenStreetMap digantikan sepenuhnya oleh lapisan ubin citra satelit asli yang bersumber dari layanan pihak ketiga *Esri World Imagery (ArcGIS)*. Foto udara ini menampilkan visualisasi nyata dari atap-atap gedung kampus Universitas Bengkulu, kepadatan tajuk pepohonan hijau hutan kampus, area lapangan rumput, struktur jalan aspal, hingga deretan rumah-rumah penduduk di sekitar batas luar area kampus secara mendetail. Di atas citra satelit yang kompleks tersebut, sistem menggambarkan lintasan rute terpendek menggunakan garis *polyline* berwarna ungu/merah muda neon (*magenta*) yang sangat kontras, memastikan jalur navigasi tetap terlihat dengan sangat jelas dan tidak tersamar oleh warna latar



belakang foto satelit yang didominasi warna hijau dan cokelat. Di sisi kiri, panel kontrol *sidebar* kembali menggunakan skema warna dasar tema terang (*light mode*) dengan latar belakang putih bersih pada kolom input, teks navigasi berwarna hitam pekat yang solid, serta bar konfirmasi di bagian bawah panel yang kini berubah warna menjadi kuning keemasan dengan teks "*Mode Siang aktif*".

#### 4.6.10 Kombinasi Detour Routing Adaptif pada Kondisi Mode Malam

Dokumentasi final ini menunjukkan pengujian tingkat lanjut yang menggabungkan fitur rute cadangan adaptif di dalam skema estetika mode malam.



#### Penjelasan Visual:

Gambar ini memvisualisasikan hasil akhir dari pengujian tingkat lanjut pada skenario perjalanan Rektorat menuju Fakultas ISIP dalam kondisi mode malam. Di atas peta gelap, pengguna melakukan tindakan pemblokiran rute pada jalur jalan utama sebelah selatan (akses langsung menuju FISIP). Sistem merespons dengan meletakkan marker melingkar merah bertuliskan "DIBLOKIR" tepat di persimpangan jalan tersebut. Secara seketika, fungsi pembaca graf pada *js/routing-detour.js* memotong koneksi antar-simpul yang melewati titik tersebut dan memaksa Algoritma Dijkstra mencari simpul alternatif. Hasil kalkulasi ulang menghasilkan rute memutar yang digambar dengan garis *polyline* biru muda cerah baru: Jalur dialihkan berbelok ke arah utara melewati kompleks laboratorium teknik, memutari area gedung bersama bagian atas, lalu turun ke selatan untuk mencapai titik akhir Fakultas ISIP dari arah belakang gedung. Pada panel kontrol *sidebar* sebelah kiri, kartu "*Rute Cadangan ke-1 Ditemukan!*" kembali muncul di atas latar belakang biru pekat. Kartu metrik tersebut menampilkan hasil analisis perbandingan delta yang sangat

efisien dan menarik, di mana indikator "JARAK" tercatat bertambah menjadi 929 m dengan keterangan sub-label merah +161m lebih panjang, namun pada kotak indikator "WAKTU", sistem menampilkan teks kuning "sama" (artinya penambahan jarak 161 meter tersebut tidak memberikan perubahan durasi menit yang signifikan pada kecepatan moda mobil), membuktikan efektivitas luar biasa dari kecerdasan logika AI pencarian jalur cadangan dalam mempertahankan efisiensi waktu perjalanan pengguna.

## **BAB V**

### **KESIMPULAN DAN SARAN**

#### **5.1 Kesimpulan**

Berdasarkan hasil pengembangan, implementasi, dan pengujian sistem navigasi KibokGO UNIB, dapat ditarik kesimpulan-kesimpulan sebagai berikut:

- **1. Implementasi Algoritma Dijkstra Berhasil:** Algoritma Dijkstra telah berhasil diimplementasikan secara murni menggunakan JavaScript di sisi klien (client-side) pada berkas js/dijkstra.js. Algoritma ini memproses model graf kampus UNIB yang direpresentasikan dalam bentuk adjacency list berbobot dan menghasilkan jalur terpendek yang terbukti optimal secara matematis pada setiap skenario pengujian.
- **2. Model Graf Kampus yang Komprehensif:** Graf jaringan jalan Kampus UNIB berhasil dimodelkan dengan 29 simpul (node) yang merepresentasikan gedung dan fasilitas utama kampus, serta 65 sisi (edge) bidirectional berbobot yang merepresentasikan jalur fisik antar-lokasi berdasarkan estimasi jarak GPS dalam meter.
- **3. Sistem Navigasi Web yang Fungsional:** KibokGO UNIB berhasil dibangun sebagai aplikasi web Single Page Application (SPA) yang sepenuhnya fungsional, mendukung tiga mode transportasi (Jalan Kaki, Motor, Mobil), menampilkan rute real-time di atas peta interaktif Leaflet.js dengan data OpenStreetMap, dan dapat diakses langsung dari browser tanpa instalasi tambahan.
- **4. Fitur Detour Routing yang Adaptif:** Sistem rute cadangan (detour routing) berhasil diimplementasikan menggunakan logika kustom yang mampu mencari dan menampilkan jalur alternatif secara dinamis ketika pengguna menandai suatu jalur sebagai terblokir, dengan batas maksimal 3 rute cadangan secara berurutan.
- **5. Transparansi Komputasi Algoritma:** Fitur log perhitungan Dijkstra real-time dan overlay visualisasi jaringan graf pada peta berhasil memberikan transparansi penuh terhadap proses komputasi algoritma, menjadikan sistem ini tidak hanya berfungsi sebagai alat navigasi praktis tetapi juga sebagai media pembelajaran visual algoritma graf.
- **6. Desain Antarmuka Modern dan Responsif:** Antarmuka web KibokGO UNIB berhasil dirancang dengan standar desain modern menggunakan konsep

Glassmorphism, dark mode, animasi micro-interaction, dan layout responsif yang optimal baik di desktop maupun perangkat mobile.

## 5.2 Saran Pengembangan

Meskipun sistem KibokGO UNIB telah berhasil diimplementasikan dan berfungsi dengan baik, terdapat beberapa saran pengembangan untuk meningkatkan kualitas dan kapabilitas sistem di masa mendatang:

### 5.2.1 Saran Jangka Pendek

- **Perluasan Graf Kampus:** Menambahkan lebih banyak node dan edge untuk mencakup seluruh jalur setapak, parkir, dan area dalam gedung kampus UNIB secara lebih komprehensif, termasuk jalur bagi pejalan kaki yang berbeda dengan jalur kendaraan.
- **Persistensi Data Kondisi Jalan:** Menambahkan fitur penyimpanan kondisi jalan yang diinput pengguna (terblokir/macet) ke localStorage atau backend sederhana, sehingga informasi kondisi jalan dapat dipertahankan dan dibagikan antar sesi.
- **Validasi Bobot Edge dengan GPS:** Melakukan verifikasi ulang bobot jarak setiap sisi graf menggunakan data pengukuran GPS lapangan yang lebih akurat untuk meningkatkan presisi estimasi jarak dan waktu tempuh.

### 5.2.2 Saran Jangka Menengah

- **Integrasi Backend dan Database:** Mengembangkan backend server (Node.js/Python Flask) dengan database PostgreSQL + PostGIS untuk menyimpan dan mengelola data graf kampus, kondisi jalan, serta riwayat rute pengguna secara persisten dan dapat diperbarui oleh admin kampus.
- **Implementasi A\* Algorithm:** Menambahkan implementasi Algoritma A\* (A-Star) sebagai alternatif Dijkstra dengan fungsi heuristik jarak Haversine untuk perbandingan performa antar algoritma pada kondisi graf yang lebih besar.
- **Notifikasi Kondisi Jalan Realtime:** Mengintegrasikan sistem notifikasi push berbasis kondisi jalan aktual kampus, misalnya penutupan jalan untuk acara wisuda atau perbaikan infrastruktur, yang dapat dikirimkan kepada pengguna secara realtime.

- **Progressive Web App (PWA):** Mengkonversi KibokGO UNIB menjadi Progressive Web App (PWA) dengan kemampuan offline mode menggunakan Service Worker, sehingga peta dan data graf tetap dapat diakses meski tanpa koneksi internet.

### 5.2.3 Saran Jangka Panjang

- **Navigasi Multi-Lantai:** Mengembangkan sistem navigasi vertikal untuk gedung bertingkat di kampus UNIB, memungkinkan pencarian rute dari satu ruangan di lantai tertentu ke ruangan lain di lantai yang berbeda melalui tangga atau lift.
- **Integrasi IoT Sensor Kepadatan:** Memasang sensor IoT (Internet of Things) di titik-titik strategis kampus untuk memantau kepadatan pejalan kaki secara realtime dan secara otomatis memperbarui bobot sisi graf berdasarkan tingkat keramaian.
- **Fitur Aksesibilitas Difabel:** Menambahkan mode navigasi khusus bagi pengguna penyandang disabilitas dengan memprioritaskan jalur yang dilengkapi ramp kursi roda dan menghindari tangga atau permukaan jalan yang tidak rata.
- **Sistem Rekomendasi Berbasis AI:** Mengintegrasikan model Machine Learning untuk memprediksi kondisi jalan dan memberikan rekomendasi rute berdasarkan pola historis penggunaan, waktu hari, dan data cuaca secara proaktif kepada pengguna.